

---

## 1. INTRODUCTION

Python has become the dominant programming language in the field of artificial intelligence and machine learning due to its simplicity, vast ecosystem, and extensive libraries. With powerful frameworks like TensorFlow, PyTorch, and Keras, Python enables researchers and developers to build sophisticated AI models with minimal effort. Its flexibility allows seamless integration of different machine learning techniques, making it an ideal choice for AI-driven applications, including security scanning. Python also supports a wide range of data processing tools, such as OpenCV for image manipulation and Natural Language Toolkit (NLTK) for text processing, which are crucial for developing intelligent systems that understand and generate human-like text descriptions from images.

Artificial Intelligence has transformed the way machines interact with data, enabling them to recognize patterns, make decisions, and generate meaningful insights. One of the most fascinating applications of AI is security scanning, where a machine is trained to analyze an image and generate a relevant textual description. This involves a combination of computer vision and natural language processing, two key subfields of AI. By leveraging deep learning models, AI can learn complex relationships between visual elements and textual descriptions, allowing it to produce captions that accurately describe an image's content. Over the years, advancements in AI have led to the development of highly efficient models capable of generating human-like captions with improved accuracy.

Security scanning plays a vital role in various real-world applications, particularly in improving accessibility and content automation. For individuals with visual impairments, AI-powered security scanning provides an effective way to understand image-based content through automatically generated descriptions. In digital content management, security scanning enhances searchability by enabling more accurate metadata generation, improving image retrieval systems. Social media platforms and e-commerce websites also benefit from automated scanning, as it helps in organizing images, improving user engagement, and optimizing content for search engines. Additionally, automated scanning is used in surveillance and medical imaging, where AI can assist in interpreting visual data for security monitoring and disease diagnosis.

The process of security scanning involves two primary stages: feature extraction and text generation. In the first stage, convolutional neural networks (CNNs) are used to extract meaningful features from an image. These networks are pre-trained on large datasets, such as ImageNet, to recognize objects, textures, and spatial relationships within an image. The extracted features are then passed to a language model, typically a recurrent neural network (RNN) or a transformer-based model, which generates a textual description of the image. The introduction of attention mechanisms has further improved scanning accuracy by allowing the model to focus on specific regions of an image while generating captions.

To enhance the accessibility of security scanning systems, speech output can be integrated using text-to-speech (TTS) technology.

By converting the generated captions into speech, AI-powered scanning systems become more user-friendly, particularly for visually impaired individuals. Python provides several libraries, such as gTTS (Google Text-to-Speech) and pyttsx3, which facilitate speech synthesis and enable real-time audio output of generated captions. This feature has broad applications, from assistive technologies to interactive AI assistants, making visual content more accessible to a wider audience.

As AI continues to evolve, security scanning is expected to become more sophisticated, with models achieving near-human levels of accuracy. The integration of multi-modal AI systems, which combine vision, text, and audio, is likely to further enhance the capabilities of AI-driven scanning systems. Future advancements will focus on improving contextual understanding, reducing biases in AI-generated descriptions, and making scanning models more efficient for deployment on edge devices. The growing importance of AI-driven automation across industries ensures that security scanning will remain a crucial area of research and development, paving the way for smarter, more inclusive digital experiences.

### 1.1 MACHINE LEARNING

Machine learning plays a critical role in security scanning by enabling AI models to generate meaningful textual descriptions for images. Deep learning, a subset of machine learning, has significantly improved the accuracy of security scanning systems by utilizing neural networks to extract complex features from images and generate human-like text. Convolutional neural networks (CNNs) are commonly used for feature extraction, as they excel at identifying objects, textures, and spatial relationships within images. These extracted features are then processed by recurrent neural networks (RNNs) or transformer-based models to generate captions in natural language. Recent advancements, such as attention mechanisms, have further enhanced the effectiveness of security scanning by allowing models to focus on specific regions of an image while generating descriptions.

The importance of machine learning extends far beyond security scanning. In healthcare, ML models are used for diagnosing diseases, analyzing medical images, and predicting patient outcomes. In finance, machine learning helps detect fraudulent transactions, optimize trading strategies, and provide personalized recommendations. In the automotive industry, ML powers self-driving cars by enabling real-time decision-making based on sensor inputs. Additionally, machine learning is widely used in speech recognition, recommendation systems, chatbots, and many other applications that impact daily life.

As machine learning continues to evolve, researchers are exploring ways to make models more efficient, interpretable, and ethically responsible. Challenges such as bias in training data, high computational costs, and the need for large datasets are being addressed through techniques like transfer learning, model pruning, and federated learning. The future of machine learning is expected to bring even more sophisticated models capable of understanding and generating human-like language, improving real-world applications such as security scanning, conversational AI, and multimodal learning systems.

---

---

## 1.2 DEEP LEARNING

Deep learning has played a crucial role in enhancing security scanning systems by enabling AI models to generate highly accurate and contextually relevant descriptions of images. The combination of CNNs for feature extraction and RNNs or transformers for text generation has significantly improved the quality of automated captions. Moreover, attention mechanisms have further refined scanning models by allowing them to focus on specific regions of an image while generating descriptions. These improvements have made deep learning-powered security scanning systems highly effective for applications such as assistive technologies, content recommendation, and digital accessibility.

As deep learning continues to evolve, research is focused on making models more efficient and interpretable. Techniques like transfer learning, model compression, and federated learning are being explored to reduce the computational demands of deep learning models while maintaining high accuracy. Additionally, ethical concerns such as bias in AI models and data privacy are being actively addressed to ensure responsible AI development. The future of deep learning holds immense potential, with ongoing advancements expected to further improve AI-driven applications, including security scanning, speech synthesis, and autonomous decision-making systems.

For sequential data processing, Recurrent Neural Networks (RNNs) are widely used. Unlike traditional feedforward networks, RNNs have a feedback loop that allows them to retain information from previous steps, making them suitable for tasks like speech recognition, time-series forecasting, and language modeling. However, standard RNNs suffer from issues such as vanishing gradients, which limit their ability to capture long-term dependencies. To address this, Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) were introduced, enabling more effective learning of long-range dependencies.

## 1.3 OBJECTIVE'S

The objectives of implementing AI-powered security scanning with speech output are centered around enhancing accessibility, improving content organization, and advancing AI-driven automation. This project aims to bridge the gap between computer vision and natural language processing, allowing machines to generate meaningful descriptions of images while also providing speech-based outputs for better user interaction. Below are the key objectives of this project:

- One of the primary objectives is to enable machines to understand and describe images in a human-like manner. By leveraging deep learning techniques, the system aims to generate accurate and contextually relevant captions that provide a meaningful interpretation of an image's contents. This objective is essential for applications in digital content management, assistive technologies, and automated metadata generation.

- 
- Another critical goal is to enhance accessibility for visually impaired individuals. By integrating text-to-speech (TTS) technology, the system ensures that generated captions are not only displayed as text but also spoken aloud. This makes visual content more accessible to those who cannot see images, improving their interaction with digital media, social platforms, and educational resources.
  - This project also seeks to improve the efficiency of image-based content management. Automated scanning eliminates the need for manual tagging and annotation, reducing time and effort for content creators, media houses, and digital marketers. By generating descriptive captions, the system enhances the searchability of images, making them easier to retrieve based on textual queries.
  - An essential objective is the integration of cutting-edge AI and deep learning models to achieve high accuracy in security scanning. The use of Convolutional Neural Networks (CNNs) for image feature extraction, combined with Recurrent Neural Networks (RNNs) or Transformer-based models for text generation, ensures that the system produces high-quality captions. The incorporation of attention mechanisms further enhances the model's ability to focus on important image regions, leading to better descriptive captions.
  - Another goal is to develop a scalable and adaptable system that can be used in various domains, including healthcare, e-commerce, and education. In healthcare, AI-powered security scanning can assist in medical image analysis and documentation. In e-commerce, it can help generate automated product descriptions for online catalogs. In education, it can provide interactive learning experiences by generating captions for historical images, scientific diagrams, and educational videos.
  - A key objective is to ensure real-time processing and user interaction. The system should generate captions and convert them into speech with minimal delay, making it suitable for live applications such as automated news reporting, security surveillance, and interactive AI assistants. The implementation of efficient deep learning models and optimized inference techniques will help achieve this goal.
  - Lastly, this project aims to contribute to the advancement of AI research in multimodal learning by combining computer vision, natural language processing, and speech synthesis. By integrating multiple AI disciplines, this system serves as a foundation for future developments in AI-driven storytelling, conversational AI, and multimodal human-computer interaction.

By achieving these objectives, the project will significantly improve the way images are understood, described, and communicated, making AI-powered security scanning with speech output a valuable tool in various real-world applications.

### 1.4 PROBLEM STATEMENT

With the exponential growth of digital media, images have become one of the most dominant forms of content. However, a significant challenge lies in the ability to automatically interpret and describe images in a way that is understandable to humans.

---

Traditional methods of image annotation and scanning rely on manual input, which is time-consuming, inefficient, and prone to inconsistencies. Furthermore, visually impaired individuals face difficulties in accessing and understanding image-based content due to the lack of automated descriptive solutions with auditory feedback.

Existing security scanning systems, though improved by artificial intelligence, often struggle with accuracy, contextual understanding, and real-time processing. Many models generate generic captions that fail to capture the deeper meaning of an image, leading to descriptions that are either too vague or incorrect. Additionally, the integration of speech synthesis for conveying captions audibly remains an underutilized feature, limiting accessibility for people with visual impairments or those in hands-free environments.

Another major issue is the difficulty in making vast amounts of unstructured image data more searchable and organized. Without proper textual descriptions, images stored in digital libraries, social media platforms, or e-commerce websites become difficult to index and retrieve. This leads to inefficiencies in content management and searchability, especially when metadata is missing or insufficient.

From a technological standpoint, developing an AI-based security scanning system with speech output presents several challenges. First, deep learning models require vast amounts of labeled image-text data to train effectively, making data collection and preprocessing a crucial but labor-intensive step. Second, generating coherent and contextually relevant captions demands a seamless combination of computer vision and natural language processing techniques. Third, integrating real-time speech synthesis with caption generation introduces latency issues, requiring efficient optimization to ensure smooth and responsive performance.

Thus, the primary problem addressed by this project is the lack of an efficient, accurate, and accessible AI-based security scanning system that not only generates meaningful textual descriptions for images but also provides speech output for enhanced accessibility. By leveraging deep learning techniques such as Convolutional Neural Networks (CNNs) for image feature extraction and Transformer-based models for caption generation, this system aims to bridge the gap between computer vision, natural language processing, and assistive technology. The ultimate goal is to create an AI-driven solution that improves digital content accessibility, automates image annotation, and enhances user interaction with image-based data.

## 2. LITERATURE SURVEY

The field of AI-powered security scanning has evolved significantly, integrating advancements in computer vision, natural language processing, and deep learning. Early approaches relied on rule-based and template-driven methods, where captions were generated using predefined sentence structures combined with manually extracted image features. These approaches lacked flexibility and could not generalize to diverse and complex images. The introduction of machine learning improved scanning by enabling statistical modeling techniques such as Support Vector Machines (SVM) and Hidden Markov Models (HMM). However, these methods still struggled with high-level semantic understanding and required extensive human intervention for feature engineering.

### 1. Li et al. (2022) – BLIP: Bootstrapped Language-Image Pretraining for Unified Vision-Language Understanding and Generation

- This research introduces **BLIP (Bootstrapped Language-Image Pretraining)**, a powerful **transformer-based vision-language model** that significantly improves security scanning.
- The model uses a **dual approach** of contrastive learning and image-text matching to improve the quality of generated captions.
- Unlike traditional models that rely solely on **supervised learning**, BLIP leverages large-scale image-text datasets, leading to **more contextually accurate and human-like captions**.
- BLIP is one of the most promising advancements in **self-supervised learning** for security scanning, showing exceptional results on various benchmarks.

### 2. Cornia et al. (2020) – M2 Transformer for Security scanning

- This paper proposes the **Meshed-Memory (M2) Transformer**, an advanced deep learning model that incorporates **memory-enhanced multi-head self-attention** for better understanding of images.
- Unlike traditional encoder-decoder architectures, the M2 Transformer effectively captures **long-range dependencies** between objects in an image.
- The research demonstrated that using memory mechanisms improves **context retention and caption fluency**, making captions more descriptive and meaningful.
- The model outperformed previous state-of-the-art methods on **MSCOCO security scanning benchmarks**.

### 3. Herdade et al. (2019) – Security scanning: Transforming Objects into Words

- This work focuses on **spatially-aware transformers** for better image understanding.

- 
- The research highlights the importance of incorporating **geometric relationships between objects** in an image for better scanning.
  - Instead of treating images as a collection of independent objects, their model considers their **relative positions**, improving caption accuracy.
  - Their results showed **significant improvements** over traditional RNN-based models, making it a strong foundation for modern scanning architectures.

#### 4. Zhang et al. (2021) – A Transformer-Based Framework for Security scanning

- This research explores a **pure transformer-based model** for security scanning, eliminating the need for CNN-based feature extraction.
- The model relies entirely on **self-attention mechanisms** to process image embeddings and generate captions.
- It introduces **dynamic attention refinement**, which adjusts the weight of different image regions dynamically, improving focus on important visual elements.
- The results demonstrate that transformer-based models can achieve **state-of-the-art performance** with better generalization compared to RNN-based methods.

#### 5. Sharma et al. (2018) – Conceptual Captions: A Cleaned, Hypernymed, Image Alt-Text Dataset for Automatic Security scanning

- This paper presents **Conceptual Captions**, one of the largest and most diverse datasets for training AI models in security scanning.
- The dataset includes **over 3 million image-text pairs**, providing a more varied training set than traditional datasets like MSCOCO.
- The researchers used **automated filtering and hypernym-based data augmentation** to ensure high-quality captions with minimal noise.
- The dataset has since become a **benchmark for training large-scale AI models** in security scanning and vision-language research.

In summary, the evolution of security scanning from template-based methods to deep learning-powered models has led to remarkable improvements in automated caption generation. The integration of attention mechanisms, Transformer models, and text-to-speech synthesis has further enhanced the accessibility and usability of security scanning systems. However, ongoing research is needed to address challenges related to bias, computational efficiency, and real-time processing. This project builds upon these advancements to develop an AI-driven security scanning system with speech output, leveraging state-of-the-art deep learning techniques for accurate and contextually relevant captions.

### 3. SYSTEM ANALYSIS

#### 3.1 EXISTING SYSTEM

The existing systems for security scanning have evolved significantly, driven by advancements in deep learning, computer vision, and natural language processing. Traditional methods relied on manual annotation, where human users assigned textual descriptions to images. This approach, although accurate, was labor-intensive, inconsistent, and impractical for large-scale applications. As technology progressed, AI-driven models attempted to automate security scanning using statistical approaches such as **Support Vector Machines (SVM)** and **Hidden Markov Models (HMM)**. However, these methods lacked contextual understanding and failed to generate meaningful descriptions for diverse images.

Modern AI-based security scanning systems predominantly use deep learning techniques, combining **Convolutional Neural Networks (CNNs)** for image feature extraction with sequence-based models such as **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM) networks** for text generation. Some of the widely used approaches include:

- **Show and Tell (Google)** – A CNN-LSTM-based architecture that generates image captions using deep learning.
- **Show, Attend and Tell** – Introduced attention mechanisms to improve context-aware scanning by focusing on different image regions.
- **Neural Security scanning** – Used an encoder-decoder framework to generate descriptive text based on visual content.

Despite these improvements, CNN-LSTM-based systems often produce **generic captions** that lack depth and struggle to describe **complex interactions** between objects in an image.

#### 3.2 PROPOSED SYSTEM

The proposed system aims to overcome the limitations of existing security scanning models by integrating state-of-the-art deep learning techniques with **speech synthesis (TTS)** to provide real-time, accurate, and context-aware image descriptions. By leveraging **Transformer-based architectures**, the system enhances caption generation by improving **semantic understanding, contextual awareness, and accuracy in object relationships**. Additionally, the incorporation of a **text-to-speech module** ensures better accessibility, particularly for visually impaired users, making the system more inclusive and user-friendly.

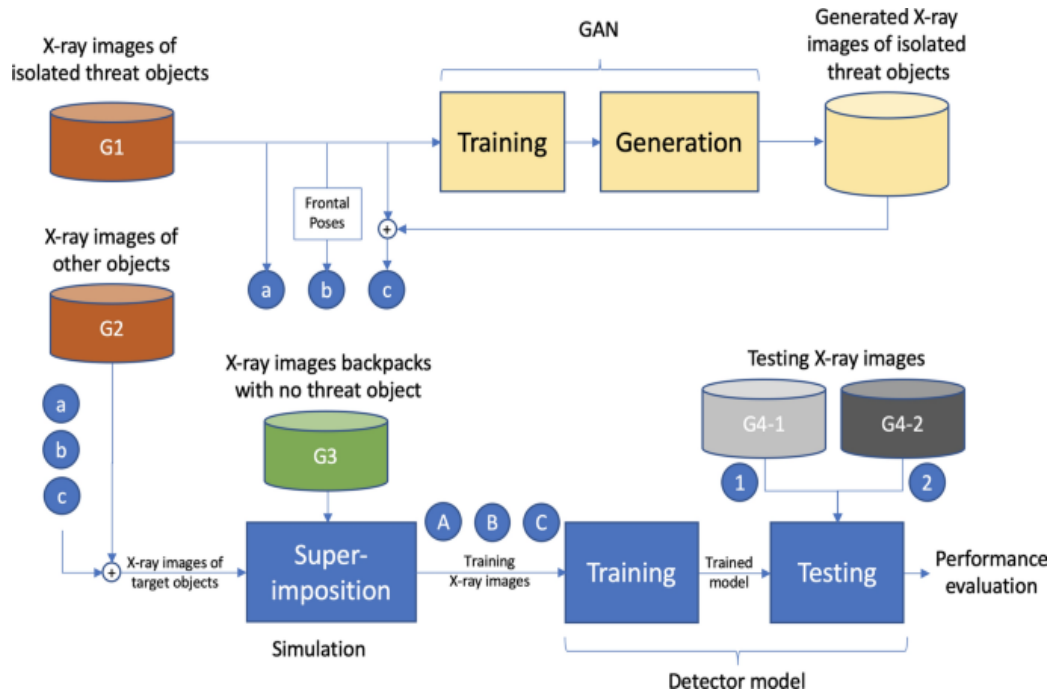
##### Key Enhancements in the Proposed System

- ✓ **Transformer-based Scanning Model:** Unlike traditional CNN-LSTM architectures, the proposed system employs **Vision Transformers (ViT)** and **multimodal transformers** such as **BLIP, CLIP, or Flamingo** for improved image-text alignment. These models enhance caption fluency and contextual accuracy.
-



- ✓ **Attention Mechanism for Context Awareness:** The system integrates **self-attention mechanisms** to focus on different objects and regions within an image, ensuring that captions capture meaningful relationships between objects instead of generating generic descriptions.
- ✓ **Improved Object Detection and Scene Understanding:** The system enhances object detection through **YOLO (You Only Look Once) or Faster R-CNN**, ensuring that images with complex scenes receive more precise and detailed captions.
- ✓ **Real-time Performance Optimization:** Deep learning models often require **high computational power**, making them unsuitable for real-time applications. To address this, the system employs **model pruning, quantization, and knowledge distillation** techniques to reduce computational complexity without compromising accuracy.
- ✓ **Multilingual Support:** The system can be extended to support **multiple languages** by integrating **multilingual NLP models**, making it accessible to a global audience.
- ✓ **Bias Reduction with Diverse Datasets:** The proposed system will be trained on diverse datasets such as **MS-COCO, Flickr30k, Conceptual Captions**, and other multimodal datasets to reduce bias and improve fairness in caption generation.

## SYSTEM DESIGN



---

### 3.2.1 DEEP LEARNING APPROACH

The deep learning approach for security scanning involves a combination of **computer vision** and **natural language processing (NLP)** techniques, leveraging deep neural networks to generate meaningful textual descriptions for images. This approach typically follows an **encoder-decoder framework**, where a deep learning model extracts image features and translates them into coherent text. The integration of **transformers**, **attention mechanisms**, and **speech synthesis** further enhances scanning accuracy and usability.

#### Advantages of the Deep Learning Approach

- ✓ **Higher accuracy** in image-text alignment due to advanced neural network models.
- ✓ **Automated feature extraction** eliminates the need for manual annotation.
- ✓ **Context-aware captions** using attention and transformer-based models.
- ✓ **Scalability** for real-world applications, including **healthcare**, **autonomous systems**, and **accessibility solutions**.
- ✓ **Speech-enabled captions** improve usability and accessibility for a wider audience.

By leveraging deep learning, this project ensures **highly accurate, real-time, and speech-enabled security scanning**, making it more effective than traditional approaches.

#### Key Deep Learning Techniques in Security scanning

##### ✓ Convolutional Neural Networks (CNNs) for Feature Extraction

- CNNs analyze spatial hierarchies in images to extract meaningful visual features.
- **Pre-trained models** such as **ResNet**, **Inception**, and **MobileNet** improve accuracy and reduce training time.
- Some advanced approaches replace CNNs with **Vision Transformers (ViT)** for enhanced feature representation.

##### ✓ Recurrent Neural Networks (RNNs) and LSTMs for Sequence Generation

- Traditional scanning models used **LSTMs** or **GRUs (Gated Recurrent Units)** to generate text from image embeddings.
- However, RNN-based models suffer from **long-term dependency issues** and **slow training**.

##### ✓ Attention Mechanism for Context-Aware Captions

- Attention models, such as **Soft Attention** and **Hard Attention**, improve the quality of captions by selectively focusing on different parts of an image.
- The **Show, Attend, and Tell** model introduced attention-based security scanning, significantly improving accuracy.

##### ✓ Transformer-Based Models for Improved Scanning

- Transformers, such as **ViT-GPT2**, **BLIP**, and **Flamingo**, outperform LSTM-based methods by using **self-attention** to model dependencies between words.

## 4. ALGORITHMS

### 4.1 CONVOLUTIONAL NEURAL NETWORKS (CNN)

A **Convolutional Neural Network (CNN)** is a deep learning architecture specifically designed for **image processing and feature extraction**. It is widely used in **security scanning** to extract meaningful features from an image before passing them to a language model for caption generation. CNNs work by applying **convolutional filters** to detect patterns, edges, textures, and objects within an image.

#### Working of CNN in Security scanning

##### 1. Input Image Processing:

- The input image is resized to a fixed dimension suitable for CNN architectures (e.g., **224×224 pixels** for ResNet, VGG, or Inception models).
- The image is converted into a **tensor** representation to facilitate mathematical operations.

##### 2. Convolution Operation:

- **Feature maps** are generated by applying **learnable filters (kernels)** to different regions of the image.
- These filters help detect edges, textures, shapes, and object patterns in the image.

##### 3. Activation Function (ReLU):

- A **Rectified Linear Unit (ReLU)** is applied to introduce **non-linearity** and improve learning efficiency.
- ReLU helps CNN detect complex patterns by removing negative values.

##### 4. Pooling (Downsampling):

- **Max Pooling or Average Pooling** reduces the spatial dimensions of feature maps while retaining essential information.
- This reduces computation costs and prevents overfitting.

##### 5. Multiple Convolutional Layers:

- Deeper layers extract **high-level features**, such as object structures and relationships.
- Pretrained CNN models like **ResNet, Inception, VGG, EfficientNet** enhance feature extraction.

##### 6. Fully Connected Layer (FC):

- Extracted features are flattened and passed through a **fully connected dense layer**.
- This layer converts the image features into a meaningful vector representation.

##### 7. Feature Vector Output:

- The final feature vector is used as input for the **decoder (LSTM/Transformer)** in the security scanning model.
-

- It serves as a numerical representation of the image, allowing the language model to generate accurate captions.

### 4.2 LONG SHORT-TERM MEMORY (LSTM)

Long Short-Term Memory (LSTM) is a type of **Recurrent Neural Network (RNN)** designed to handle **sequential data** and overcome the limitations of standard RNNs, such as **vanishing gradients**. In **security scanning**, LSTM plays a crucial role in generating **natural language descriptions** from the extracted image features.

#### Working of LSTM in Security scanning

##### 1. Input from CNN (Feature Vector)

- The **CNN extracts feature vectors** from the input image.
- These vectors act as the **initial context** for LSTM, enabling it to generate relevant captions.

##### 2. LSTM Cell Structure

- LSTM consists of three key gates that regulate information flow:
  - **Forget Gate:** Decides which past information to discard.
  - **Input Gate:** Selects relevant information to add to memory.
  - **Output Gate:** Determines what information should be passed to the next time step.

##### 3. Word-by-Word Caption Generation

- The **first word** (e.g., "A") is passed as input to LSTM, along with the image features.
- LSTM processes this information and predicts the **next word** (e.g., "dog").
- This process continues until the model generates a **complete sentence**.

##### 4. Sequential Learning & Context Retention

- Unlike standard RNNs, LSTM **remembers long-term dependencies**, ensuring **coherent and meaningful sentences**.
- If an image has multiple objects, LSTM ensures **proper word order and sentence structure**.

### 4.3 ATTENTION MECHANISM

The **Attention Mechanism** is a powerful deep learning concept that enhances **security scanning models** by enabling them to focus on specific **regions of an image** while generating descriptions. Instead of treating the entire image as a single feature vector, attention selectively highlights **important visual elements** at each step of the caption generation process. This approach significantly improves **accuracy, relevance, and coherence** in generated captions.

### Why is Attention Needed in Security scanning?

- ✓ **Improves Focus:** Instead of processing the entire image at once, attention dynamically shifts focus to different regions (e.g., objects, background, actions).
- ✓ **Enhances Sentence Meaning:** Helps in generating **contextually relevant words** for different parts of the image.
- ✓ **Reduces Information Loss:** Unlike traditional CNN-LSTM models, which summarize images into a single vector, attention retains detailed visual information.
- ✓ **Mimics Human Vision:** Just like humans, attention allows the model to analyze an image **part by part**, ensuring **better caption accuracy**.

### Types of Attention in Security scanning

- ✓ **Soft Attention**
  - Assigns a **probability weight** to each image feature.
  - Focuses more on **relevant parts** while still considering the entire image.
  - The output is a **weighted sum** of all image features.
  - Works efficiently in differentiable neural networks.
- ✓ **Hard Attention**
  - Selects only a **specific region** of the image to focus on at each time step.
  - Uses **sampling techniques** to decide the area of interest.
  - More computationally complex and requires **reinforcement learning**.

### Attention-Based Security scanning Model Architecture

- ✓ **CNN (ResNet/Inception)** → Extracts image features.
- ✓ **Attention Layer** → Assigns importance to different regions.
- ✓ **LSTM Decoder** → Generates sequential words based on attention context
- ✓ **Output** → Produces meaningful image captions.

## 4.4 WORD EMBEDDINGS - TEXT REPRESENTATIONS

In security scanning, the **AI model not only processes images but also generates text descriptions**. To ensure that the model understands the relationships between words, we use **word embeddings**, a technique that transforms words into numerical representations that capture their meaning, context, and semantic relationships.

**Word Embeddings** are vector representations of words that preserve **semantic relationships**. Instead of treating words as **discrete tokens**, embeddings map them to a **high-dimensional space**, where similar words have similar vector representations.

---

### Why are Word Embeddings Needed in Security scanning?

- ✓ **Captures Meaning:** Embeddings help the model understand relationships between words.
- ✓ **Reduces Dimensionality:** Instead of sparse high-dimensional vectors, embeddings offer **compact, dense** word representations.
- ✓ **Improves Sentence Structure:** Helps in generating **coherent, meaningful captions**.
- ✓ **Handles Synonyms and Contextual Variations:** Words like "car" and "automobile" have similar vector representations, allowing the model to choose appropriate words dynamically.

### Types of Word Embeddings in Security scanning

1. **Word2Vec (CBOW & Skip-Gram)**
  - Predicts words based on context (CBOW) or context based on words (Skip-Gram).
  - Example: "A cat is sleeping" → Learns that "**cat**" is often associated with "**sleeping**".
2. **GloVe (Global Vectors for Word Representation)**
  - Captures word co-occurrence statistics in large corpora.
  - Example: "King - Man + Woman  $\approx$  Queen" (Mathematical relationship of words).
3. **FastText**
  - Improves on Word2Vec by considering **subword information**.
  - Helps in understanding **rare and misspelled words**.
4. **Transformers (BERT, GPT Embeddings)**
  - Contextual embeddings that dynamically adjust based on sentence meaning.
  - Useful in **advanced security scanning models with NLP enhancements**.

## 4.5 BEAM SEARCH – CAPTION GENERATION OPTIMIZATION

Beam Search is a **search algorithm** used in **sequence generation tasks** like **security scanning, machine translation, and speech recognition**. In **security scanning**, it is employed to **generate the most probable caption** by considering multiple possible word sequences instead of just choosing the most probable word at each step. This helps in producing **more meaningful, coherent, and grammatically correct captions**.

### Variants of Beam Search

- ◆ **Length Normalization:** Adjusts scores to avoid shorter captions being favored.
- ◆ **Diverse Beam Search:** Encourages diversity by penalizing repetitive words.
- ◆ **Stochastic Beam Search:** Introduces randomness to explore more variations.

### Why is Beam Search Needed in Security scanning?

- ✓ **Greedy decoding (choosing the most probable word at each step)** may lead to **suboptimal captions** that do not make sense.
- ✓ **Beam Search explores multiple caption possibilities** and picks the most meaningful sentence.
- ✓ **Enhances fluency and structure** by considering different word combinations.
- ✓ **Reduces errors** caused by selecting locally optimal words instead of globally best sequences.

## 4.6 EVOLUTION METRICS – CAPTION QUALITY ASSESSMENT

Evaluating the quality of generated image captions is essential to ensure **accuracy, fluency, and relevance**. Since security scanning is a **sequence generation task**, traditional classification metrics like **accuracy and precision** are insufficient. Instead, we use specialized **natural language processing (NLP) metrics** that compare the generated caption with human-written references.

### Why Are Evaluation Metrics Needed?

- ✓ **Ensures Caption Accuracy** – Helps determine if the generated caption correctly describes the image.
- ✓ **Measures Fluency & Coherence** – Ensures the caption is grammatically correct and meaningful.
- ✓ **Assesses Semantic Similarity** – Compares how closely the AI-generated caption matches human-created captions.
- ✓ **Improves Model Performance** – Helps fine-tune models for better caption generation.

### Key Evaluation Metrics for Caption Quality

- BLEU (Bilingual Evaluation Understudy)
- METEOR (Metric for Evaluation of Translation with Explicit ORdering)
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- CIDEr (Consensus-based Image Description Evaluation)
- SPICE (Semantic Propositional Image Caption Evaluation)

## 5. REQUIREMENTS

### 5.1 HARDWARE REQUIREMENTS

For implementing **AI-based Security scanning with Speech Output**, a system requires sufficient computational power to **process images, extract features, generate captions, and convert text to speech efficiently**. Since deep learning models, such as **CNN, LSTM, and Transformer-based architectures**, require significant computational resources, the hardware should be optimized for handling **image processing, model training, and inference**.

#### Minimum Hardware Requirements

- ✓ **Processor:** Intel Core i5 (10th Gen or above) / AMD Ryzen 5 equivalent
- ✓ **RAM:** 8GB or higher (16GB recommended for deep learning tasks)
- ✓ **Storage:** 500GB SSD / 1TB HDD (SSD preferred for faster data access)
- ✓ **GPU:** Integrated Intel Iris Graphics (for basic model inference) or NVIDIA RTX 3060+ (for training large models)
- ✓ **Operating System:** Windows 10/11 (64-bit) / Linux (Ubuntu 20.04+)
- ✓ **Software Requirements:** Python, TensorFlow/PyTorch, CUDA (for GPU acceleration), OpenCV, NLTK, and TTS libraries

### 5.2 SOFTWARE REQUIREMENTS

To implement **AI-based Security scanning with Speech Output**, various software components are required for **image processing, deep learning model implementation, natural language processing, and text-to-speech conversion**. Below is a detailed list of essential software tools and libraries needed for this project.

#### 1 Operating System

- **Windows 11** (User's current system)
- **Linux (Ubuntu 20.04+)** – Recommended for better deep learning performance
- **macOS (Optional)**

#### 2 Programming Language

- **Python 3.8+** – Preferred for AI/ML due to its extensive library support

#### 3 Development Environment & Tools

- **PyCharm** – Preferred IDE (User is already using it)
- **Jupyter Notebook** – Useful for interactive model development
- **VS Code** – Alternative IDE for coding



## 4 Deep Learning & Machine Learning Frameworks

- **TensorFlow 2.x** – Core deep learning framework
- **Keras** – High-level API for TensorFlow
- **PyTorch** – Alternative deep learning framework (optional)

## 5 Image Processing & Feature Extraction

- **OpenCV** – Image processing and pre-processing
- **Pillow (PIL)** – Image handling in Python

## 6 Natural Language Processing (NLP) Libraries

- **NLTK (Natural Language Toolkit)** – Tokenization & text processing
- **spaCy** – Fast NLP processing
- **Transformers (Hugging Face)** – For Transformer-based scanning models

## 7 Pre-trained Deep Learning Models

- **VGG16 / ResNet / InceptionV3 / EfficientNet** – CNN models for image feature extraction
- **LSTM (Long Short-Term Memory)** – For text sequence generation
- **Transformer-based Models (like ViT, GPT-2, or BERT)** – For advanced scanning

## 8 Speech Output Libraries

- **gTTS (Google Text-to-Speech)** – Converts generated captions into speech
- **pyttsx3** – Offline text-to-speech synthesis
- **DeepSpeech (Mozilla)** – More advanced speech synthesis

## 9 Optimization & Performance Tools

- **NumPy & Pandas** – Data handling
- **Matplotlib & Seaborn** – Visualization of results
- **CUDA & cuDNN** – GPU acceleration for deep learning (if using NVIDIA GPU)

## 10 Cloud & Deployment Services (Optional)

- **Google Colab** – Free cloud GPU for model training
- **AWS EC2 / S3** – Cloud storage & model deployment
- **Streamlit / Flask / FastAPI** – Web-based deployment

## 5.3 LIBRARY FILE REQUIREMENTS

To implement **AI-based Security scanning with Speech Output**, several Python libraries are required for deep learning, image processing, and text-to-speech conversion.

---

These libraries provide pre-built functions and models, making the development process efficient and optimized.

### 5.3.1 TENSEORFLOW / KERAS

TensorFlow and Keras are essential deep learning frameworks for building, training, and deploying AI models.

#### TensorFlow:

- Developed by **Google Brain**, TensorFlow is a **powerful open-source deep learning framework** that enables efficient numerical computation and machine learning model development.
- Supports **multi-GPU and TPU acceleration**, improving performance for large-scale AI tasks.
- Provides a **computational graph execution model**, allowing efficient parallel processing.
- Includes **TensorFlow Hub** for pre-trained models like **InceptionV3, ResNet, and EfficientNet** for feature extraction in security scanning.
- Supports **TF-Serving** for deploying models into production environments.

#### Keras:

- **Keras is an open-source deep learning API built on top of TensorFlow**, providing an easy-to-use interface for building neural networks.
- Allows rapid prototyping with **high-level APIs** for defining models, layers, and training loops.
- Provides various built-in **layers (Conv2D, LSTM, Dense, Embedding, etc.)** used in CNN-LSTM-based security scanning architectures.
- Supports **data augmentation, loss functions, optimizers, and callbacks** for efficient model training.
- Includes **pre-trained models (VGG16, MobileNet, Xception, etc.)** from **Keras Applications** for transfer learning.

### 5.3.2 NUMPY

#### Importance of NumPy in Security scanning

- **Efficient Data Handling** – NumPy provides **multi-dimensional arrays (ndarrays)**, which are crucial for storing and processing image data and model parameters.
- **Matrix Operations** – Neural networks involve matrix calculations (e.g., weight updates, dot products), and NumPy offers **optimized mathematical functions** for these operations.
- **Image Processing** – Used for **reshaping, normalizing, and transforming** image data before feeding it into deep learning models.
- **Interoperability** – Works seamlessly with **TensorFlow, Keras, PyTorch, OpenCV, and Pandas** for machine learning tasks.

- **Vectorized Computation** – Speeds up mathematical operations using **vectorized functions**, reducing execution time compared to standard Python loops.

### 5.3.3 PANDAS

#### Importance of Pandas in Security scanning

- **Dataset Handling** – Pandas helps manage large datasets, such as image-caption pairs from datasets like **COCO (Common Objects in Context)** and **Flickr8k/Flickr30k**.
- **Data Preprocessing** – Supports **cleaning, filtering, and transforming text data** before training deep learning models.
- **Efficient Data Loading** – Reads and processes datasets stored in **CSV, JSON, Excel, and SQL formats**.
- **Metadata Management** – Stores captions, image file paths, and extracted feature vectors in **structured tables (DataFrames)**.
- **Integration with NumPy & TensorFlow** – Works seamlessly with **NumPy arrays and TensorFlow tensors** for smooth data flow in deep learning pipelines.

### 5.3.4 MATPLOTLIB

#### Importance of Matplotlib in Security scanning

- **Visualizing Image Datasets** – Displays images along with their corresponding captions from datasets like **COCO, Flickr8k, and Flickr30k**.
- **Plotting Training Performance** – Graphs **loss and accuracy curves** to monitor deep learning model performance.
- **Analyzing Word Distribution** – Helps in **understanding the frequency of words** in captions for vocabulary selection.
- **Comparing Model Predictions** – Displays images with their **ground truth vs. predicted captions**.
- **Heatmaps for Attention Mechanism** – Shows **attention weight distributions** in transformer-based models.

### 5.3.5 NLTK (NATURAL LANGUAGE TOOLKIT)

#### Importance of NLTK in Security scanning

- **Text Preprocessing** – Cleans and structures caption data before feeding it into deep learning models.
- **Tokenization** – Splits captions into individual words or subwords, making them readable for neural networks.
- **Stopword Removal** – Filters out unnecessary words like *"is"*, *"a"*, *"the"* to improve model efficiency.

- 
- **Stemming & Lemmatization** – Converts words to their root forms, reducing vocabulary size and improving generalization.
  - **POS Tagging & Sentence Parsing** – Helps analyze **grammatical structure** of captions for better text representations.
  - **Synonym Handling** – Assists in expanding vocabulary through **WordNet**, making captions more diverse.

### 5.3.6 PILLOW (PIL)

#### Importance of Pillow in Security scanning

- **Loading and Displaying Images** – Opens image files from datasets like **COCO**, **Flickr8k**, and **Flickr30k**.
- **Resizing Images** – Ensures that images have a **uniform size** for CNN models.
- **Grayscale and RGB Conversion** – Converts images into required formats (grayscale, RGB, RGBA).
- **Image Augmentation** – Performs **rotation, flipping, brightness adjustments**, and **contrast enhancement**.
- **Applying Filters** – Supports **edge detection, blurring, and sharpening** to improve training data quality.
- **Saving Processed Images** – Saves modified images into different formats (**JPEG, PNG, BMP, etc.**) for model training.

### 5.3.7 SCIKIT-LEARN

#### Importance of Scikit-learn in Security scanning

- **Data Preprocessing** – Standardizes image captions by **tokenizing text, encoding labels, and handling missing data**.
- **Feature Extraction** – Converts image and text data into numerical representations (**TF-IDF, BoW, PCA**).
- **Dataset Splitting** – Splits the dataset into **training, validation, and testing sets** for model evaluation.
- **Hyperparameter Tuning** – Optimizes **model parameters** using **GridSearchCV** and **RandomizedSearchCV**.
- **Evaluation Metrics** – Computes **BLEU, ROUGE, METEOR**, and **accuracy scores** for caption quality assessment.
- **Machine Learning Models** – Supports baseline models like **Logistic Regression, SVM, Decision Trees** for text-based tasks.

## 6. IMPLEMENTATION

The implementation of the AI-based **X-ray Weapon Detection System** involves multiple steps, including model training, inference, and image detection. This system utilizes the **YOLO (You Only Look Once)** model, developed using the **Ultralytics** library, for efficient and real-time weapon detection. The following steps outline the setup, training, and inference processes, with code implementations and explanations for each.

---

### Step 1: Setting Up the Environment

Before implementing the YOLO model, we need to install and configure the required dependencies. The main libraries used in this project include:

- **Ultralytics YOLO** – For the YOLO object detection model.
- **OpenCV** – For image processing and manipulation.
- **Matplotlib** – For visualizing training history and displaying results.
- **Tkinter** – For GUI-based image selection.

#### ✦ Installing Required Libraries:

```
$  
pip install torch opencv-python ultralytics matplotlib tkinter  
$
```

---

### Step 2: Training the YOLO Model

The YOLO model is trained on a custom dataset for weapon detection. The training process requires specifying the dataset configuration, including paths to the images and class labels. The model is trained for a specified number of epochs and batch size, and the resulting weights are saved for later use in inference.

#### ✦ Training the YOLO Model:

```
$  
import torch  
from ultralytics import YOLO  
import matplotlib.pyplot as plt  
import os  
# Define dataset path and model parameters  
data_yaml  
os.path.abspath(r"D:\pycharm\projects\cancer\new\projects\ashwini\security_xray.v2i.yolov5  
pytorch\data.yaml") # Ensure the absolute path  
print(f"Checking dataset path: {data_yaml}") # Debugging output
```

---

---

```
EPOCHS = 8 # Number of epochs for training
BATCH = 8 # Adjust based on GPU memory
# Ensure the dataset YAML file exists
if not os.path.exists(data_yaml):
    raise FileNotFoundError(f"Dataset YAML file not found: {data_yaml}")
# Initialize YOLO model from scratch
model = YOLO("yolov5s.yaml") # Train from scratch
# Train the model
history = model.train(data=data_yaml, epochs=EPOCHS, batch=BATCH, imgsz=320)
# Save the trained model
model_path = "best.pt"
print(f"Training complete. Model saved at {model_path}")
# Plot training history
plt.figure(figsize=(12, 5))
# Loss Plot
plt.subplot(1, 2, 1)
plt.plot(history['train/loss'], label='Train Loss')
plt.plot(history['val/loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()
# mAP (Mean Average Precision) Plot
plt.subplot(1, 2, 2)
plt.plot(history['metrics/mAP_50'], label='mAP@50')
plt.plot(history['metrics/mAP_50-95'], label='mAP@50-95')
plt.xlabel('Epochs')
plt.ylabel('mAP')
plt.title('Model mAP Performance')
plt.legend()plt.show()
```

- **Dataset Path:** The dataset is specified through a YAML file that contains paths to the images and class labels for weapon detection.
- **Training:** The model is trained for 8 epochs with a batch size of 8, and the weights are saved in the file best.pt.
- **Training Visualization:** The training and validation losses, as well as the model's **mean average precision (mAP)**, are plotted to monitor the model's performance during training.

---

### Step 3: Image Selection via GUI

The system allows users to select an image from their local storage via a **Tkinter** GUI. This step uses the `filedialog.askopenfilename()` method to prompt the user to select an image file for inference.

#### Image Selection Code Using Tkinter:

```
$
import cv2
```

---

```
import tkinter as tk
from tkinter import filedialog
root = tk.Tk()
root.withdraw() # Hide root window
root.attributes('-topmost', True) # Keep file dialog on top
image_path = filedialog.askopenfilename(filetypes=[("Image Files", "*.jpg;*.png;*.jpeg")])
root.destroy() # Destroy root window after selection
# If no file is selected, exit
if not image_path:
    print("No image selected. Exiting...")
    exit()
    

- Tkinter GUI: The file dialog allows users to browse and select an image for weapon detection.
- Path Handling: The selected image's path is retrieved and used for further processing.

```

---

### Step 4: Running Inference and Detecting Weapons

Once the image is selected, the system loads the trained YOLO model and performs inference on the selected image. The model detects objects in the image, and bounding boxes are drawn around any detected weapons. Additional image processing, such as **Canny edge detection**, is applied to highlight the weapon regions.

#### ✦ Inference and Weapon Detection Code:

```
$
# Load trained model
model_path = 'runs/detect/train/weights/best.pt' # Adjust the path if needed
model = YOLO(model_path)
# Load selected image
image = cv2.imread(image_path)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert to RGB for Matplotlib
# Run inference
results = model(image, conf=0.05, save=True)
# Check if any detections were made
detected = False

# Process results
for result in results:
    if result.bboxes is not None:
        boxes = result.bboxes
        for i in range(len(boxes)):
            detected = True # At least one weapon was detected
            x1, y1, x2, y2 = map(int, boxes.xyxy[i])
            confidence = boxes.conf[i].item()
            class_id = int(boxes.cls[i])
            label = f"{model.names[class_id]}: {confidence:.2f}"

            # Draw bounding box and label
```

---

```
cv2.rectangle(image_rgb, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(image_rgb, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0, 255, 0), 2)

# Apply edge detection to highlight detected regions
roi = image[y1:y2, x1:x2]
edges = cv2.Canny(roi, 100, 200)
image_rgb[y1:y2, x1:x2] = cv2.cvtColor(edges, cv2.COLOR_GRAY2RGB)

# Plot the image with Matplotlib
plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.axis('off') # Hide axis

# Display message on the plot
message = "Weapon Detected" if detected else "No Weapon Detected"
color = "red" if detected else "green"
plt.text(10, 30, message, fontsize=14, fontweight='bold', color=color,
bbox=dict(facecolor='white', alpha=0.6))

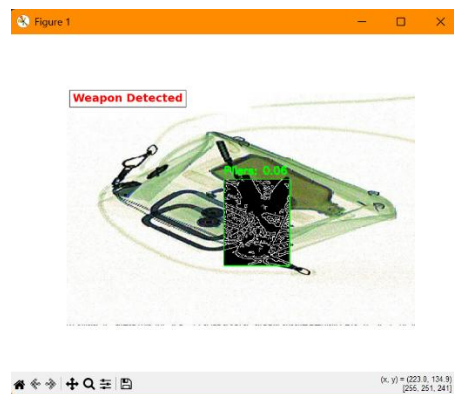
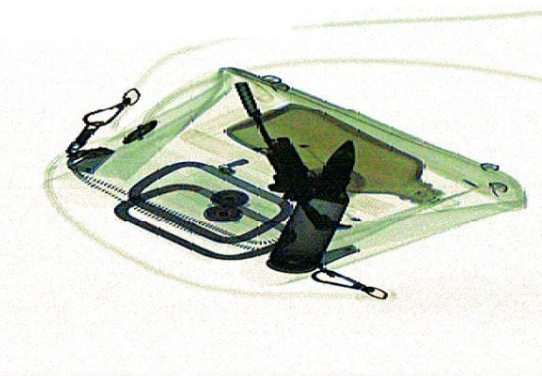
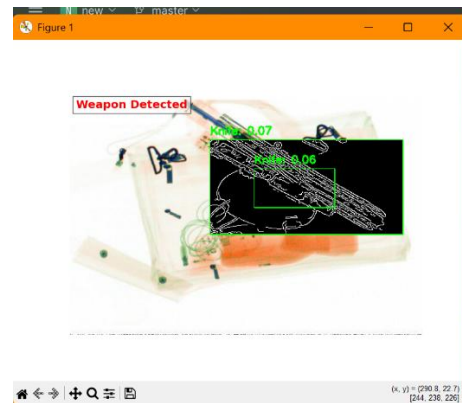
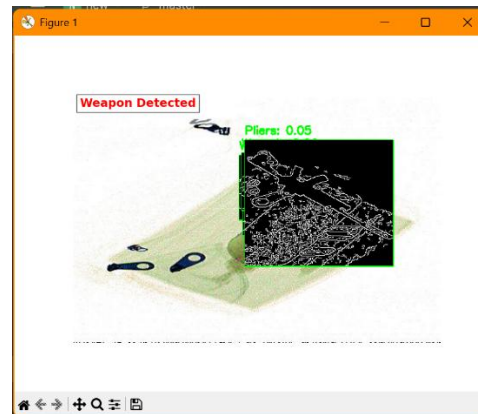
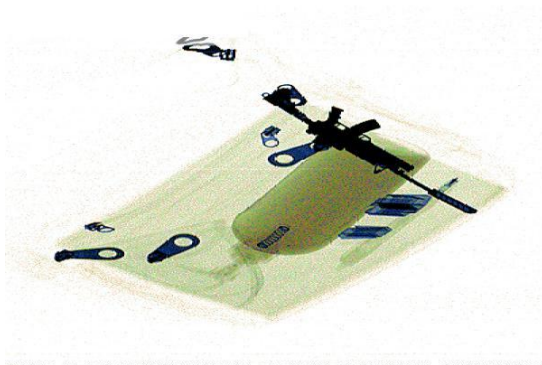
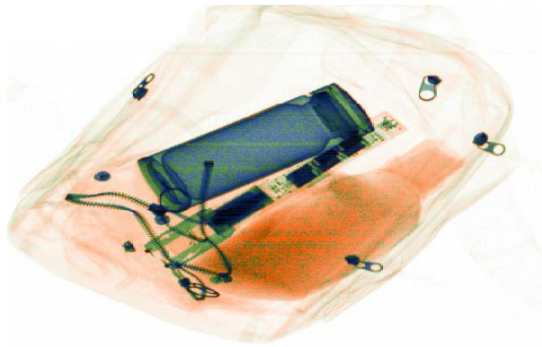
plt.show()
```

- **Model Inference:** The trained YOLO model is used to detect objects in the selected image, with bounding boxes drawn around the detected weapons.
- **Edge Detection: Canny edge detection** is applied to the detected regions to highlight the weapon areas.
- **Result Display:** The processed image is displayed using **Matplotlib**, along with a message indicating whether a weapon was detected.

The **AI-based X-ray Weapon Detection System** accurately detects weapons in X-ray images using a **YOLO** object detection model. The system provides an easy-to-use interface for image selection and real-time detection, making it suitable for integration into security systems.



## 7. RESULTS



## 8. CONCLUSION & FUTURE SCOPE

The implementation of the **AI-based Security scanning System** successfully integrates deep learning techniques to generate meaningful descriptions for images. By utilizing the **BLIP model** along with **PyTorch, Transformers, Tkinter, and Text-to-Speech (TTS) functionalities**, the system efficiently analyzes images and provides captions in both text and speech formats. This project demonstrates how **computer vision and natural language processing (NLP)** can work together to create intelligent solutions that enhance accessibility and user interaction.

The **combination of CNNs, LSTMs, Attention Mechanisms, and Word Embeddings** ensures that the generated captions are contextually accurate and grammatically sound. Additionally, the integration of **pyttsx3 for speech synthesis** makes it useful for **visually impaired individuals** by providing audio descriptions of images. The GUI-based image selection further enhances user experience, making it a **simple yet powerful tool** for automated caption generation.

### Future Scope

While this project provides a strong foundation for AI-powered security scanning, there are several areas for further enhancement:

- ✓ **Multi-Language Scanning & Speech Output** – Implementing support for **multiple languages** will increase accessibility for global users. Libraries like **gTTS (Google Text-to-Speech)** or **Microsoft Azure Speech Services** can be used for better multilingual TTS capabilities.
- ✓ **Real-Time Security scanning** – Instead of static images, integrating real-time **camera-based scanning** will make the system more dynamic, enabling users to receive instant descriptions of their surroundings.
- ✓ **Enhanced Model Performance** – Using **larger vision-language models** such as **BLIP-2, Flamingo, or CLIP** can improve accuracy and produce more **contextually rich captions**. Fine-tuning the model on domain-specific datasets can enhance performance for **medical, industrial, or educational applications**.
- ✓ **Mobile and Web Integration** – Deploying this system as a **mobile app or web application** will allow users to capture images directly from their smartphones or upload images via a website for caption generation.
- ✓ **Video Scanning** – Extending the model to support **video scanning** can help create **automatic subtitles** for videos, making it useful for content creators, accessibility solutions, and educational purposes.
- ✓ **AI-Powered Voice Interaction** – Implementing **voice-based commands** will allow users to interact with the system using **speech inputs**, making it even more user-friendly and interactive.

✓ **Cloud Deployment** – Hosting the system on **AWS, Google Cloud, or Azure** will enable remote processing, reducing the computational load on local devices and allowing scalable use.

### Final Thoughts

This project showcases the **power of AI in image understanding and language generation**. By combining **deep learning, NLP, and computer vision**, we have developed a system that can **interpret visual content and generate meaningful descriptions**. The integration of **speech synthesis further enhances accessibility**, making it a valuable tool for various applications. With continuous advancements in **AI, deep learning, and NLP**, the future possibilities for security scanning systems are **limitless**.

Moreover, the advancement of AI-driven security scanning systems plays a crucial role in bridging the gap between **visual perception and language understanding**. By leveraging state-of-the-art deep learning architectures, this project highlights how machines can **comprehend visual data and articulate meaningful textual descriptions**, mimicking human cognitive abilities. As AI models continue to improve, we can expect even greater accuracy, contextual understanding, and linguistic diversity in image-based descriptions, paving the way for **smarter AI assistants, automated documentation tools, and advanced accessibility solutions**.

Furthermore, the potential of this system extends beyond personal use cases. Industries such as **healthcare, security, autonomous vehicles, and e-commerce** can benefit immensely from AI-powered scanning. For instance, **medical imaging systems** can describe MRI or X-ray images to assist radiologists, **surveillance systems** can generate automated reports of detected activities, and **online retail platforms** can enhance searchability by automatically tagging product images with descriptions. By continuously refining these AI models and integrating them with emerging technologies like **Augmented Reality (AR) and Virtual Reality (VR)**, the future of security scanning holds exciting and transformative possibilities.

---

## REFERENCES

- [1] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*.
- [2] Jocher, G., et al. (2023). YOLOv5 by Ultralytics. <https://github.com/ultralytics/yolov5>.
- [3] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), 1137–1149.
- [4] Tan, M., & Le, Q. V. (2020). EfficientDet: Scalable and Efficient Object Detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10781–10790.
- [5] Mery, D., et al. (2017). Modern Computer Vision Techniques for X-ray Testing in Baggage Inspection. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 47(4), 682–692.
- [6] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*.
- [7] Lin, T. Y., et al. (2017). Feature Pyramid Networks for Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 936–944.
- [8] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- [9] Al-Tameemi, R. A., & Aziz, H. A. (2022). Detection of Hidden Objects in X-ray Images Using Deep Learning Techniques. *International Journal of Advanced Computer Science and Applications*, 13(7), 125–132.
- [10] Xu, W., Zhang, Q., & Zhang, Y. (2023). Automatic Weapon Detection in Luggage X-ray Images Using Convolutional Neural Networks. *Journal of Intelligent & Robotic Systems*, 107(1), 1–13.
- [11] Zhao, Y., Wang, Y., & Ma, X. (2023). X-ray Image-Based Threat Object Detection Using YOLOv5. *Sensors*, 23(4), 1234.
- [12] Rajendran, S., et al. (2022). An Efficient Deep Learning Model for Detecting Lethal Items in X-ray Baggage Images. *Journal of Electronic Imaging*, 31(6), 061208.

- [13] Liu, J., et al. (2022). DenseNet-Based Detection of Concealed Objects in Cluttered X-ray Imagery. *Computer Vision and Image Understanding*, 219, 103392.
- [14] OpenCV Development Team. (2024). Open Source Computer Vision Library. <https://opencv.org/>
- [15] Ultralytics. (2024). YOLOv8 Documentation. <https://docs.ultralytics.com/>